2600
2600, école de cybersécurité

MODULE
Noyau Windows

TP 02

Driver Echo & MiniFilter

Communication kernel/user et filtrage d'accès fichiers

AUTEUR
Lionel Auroux

DATE
2026

DURÉE
5h

Sommaire

Objectifs 3
Prérequis 3
Rendu attendu 3
Exercice 1 : Echo Driver (2h30) 4
Contexte 4
Objectifs 4
Indications 5
Architecture du driver 5
Dispatch Function 5
Programme userland 5
Chargement et test 5
Vérification avec WinDBG 5
Exercice 2 : MiniFilter (2h) 6
Contexte 6
Objectifs 6
Indications 6
Structure d'un minifilter 6
Récupérer le nom du fichier 7
Fichier .reg 7
Chargement 7
Vérification avec WinDBG 7
Exercice 3 (bonus) : Minifilter bloquant (30min) 7
Barème indicatif 8
Troubleshooting 8

Objectifs

L'objectif de cet atelier est de développer des drivers Windows complets avec communication kernel/user :

- Un driver Echo avec DeviceIoControl
- Un MiniFilter loggant les accès fichiers



Fig. 1. – width:400px

Prérequis

- TP 01 complété : VM configurée, WinDBG fonctionnel, premier driver compilé et chargé
- Offsets de `_EPROCESS` notés (ActiveProcessLinks, Token, ImageFileName)
- Visual Studio + WDK opérationnels

Rendu attendu

À la fin de ce TP, vous devez rendre une archive `tp02_NOM_PRENOM.zip` contenant :

Fichier	Description
`rapport_tp02.md`	Réponses aux questions, captures d'écran, observations

`echo/echo.c`	Code source du driver Echo
`echo/echo.h`	Header partagé kernel/userland
`echo/user_echo.c`	Programme userland
`filter/filter.c`	Code source du minifilter
`filter/filter.reg`	Fichier registre du minifilter

Le rapport doit contenir les réponses à toutes les questions numérotées et des captures d'écran de DbgView/WinDBG montrant le fonctionnement.

Exercice 1 : Echo Driver (2h30)

Contexte

Vous allez implémenter un driver complet exposant un device dans le namespace global, ainsi qu'un programme userland capable de communiquer avec ce driver via `DeviceIoControl`.

Objectifs

Vous devez implémenter de zéro (sans copier-coller le code du cours) :

1. `echo.h` — Header partagé contenant la définition du `CTL_CODE` :

```c
#define IOCTL_ECHO CTL_CODE(FILE_DEVICE_UNKNOWN, \
                            0x800,               \
                            METHOD_BUFFERED,      \
                            FILE_ANY_ACCESS)
```

2. `echo.c` — Un driver kernel qui :
   - Crée un device `\Device\EchoDriver` et un lien symbolique `\??\EchoDriverLink`
   - Gère les IRP `IRP_MJ_CREATE`, `IRP_MJ_CLOSE` et `IRP_MJ_DEVICE_CONTROL`
   - Pour l'IOCTL `IOCTL_ECHO` : lit les données du buffer d'entrée, les affiche via `DbgPrint`, puis les recopie dans le buffer de sortie (comportement echo)
   - Implémente un `DriverUnload` propre (suppression du lien symbolique et du device)
3. `user_echo.c` — Un programme userland qui :
   - Ouvre un handle vers `\\.\EchoDriverLink` via `CreateFileW`
   - Envoie une chaîne de caractères (passée en argument) au driver via `DeviceIoControl`
   - Affiche la chaîne retournée par le driver sur la console

Q1 : Pourquoi utilise-t-on `METHOD_BUFFERED` dans le `CTL_CODE` ? Quelles sont les autres méthodes possibles et quels sont leurs risques en termes de sécurité ?

Indications

Architecture du driver

Votre `DriverEntry` doit réaliser 4 opérations dans cet ordre :

1. Créer le device avec `IoCreateDevice` (type `FILE_DEVICE_UNKNOWN`)
2. Créer le lien symbolique avec `IoCreateSymbolicLink`
3. Enregistrer les handlers dans `DriverObject->MajorFunction[]`
4. Enregistrer `DriverUnload`

En cas d'échec d'une étape, nettoyer les ressources déjà créées avant de retourner l'erreur (pattern classique de gestion d'erreurs kernel).

Dispatch Function

Votre dispatch function reçoit un `PIRP` et doit :

- Récupérer le `IO_STACK_LOCATION` via `IoGetCurrentIrpStackLocation`
- Dispatcher selon `stackLocation->MajorFunction`
- Pour `IRP_MJ_DEVICE_CONTROL` : accéder au buffer via `Irp->AssociatedIrp.SystemBuffer`
- **Toujours** terminer par `IoCompleteRequest(Irp, IO_NO_INCREMENT)` et retourner le status

Q2 : Que se passe-t-il si vous oubliez d'appeler `IoCompleteRequest` ? Quel comportement observerez-vous côté userland ?

Programme userland

- Ouvrir le device avec `CreateFileW(L"\\\\.\\EchoDriverLink", ...)`
- Utiliser `DeviceIoControl` avec les buffers d'entrée et de sortie
- Compiler avec : `cl user_echo.c /I . /Fe:user_echo.exe`

Chargement et test

```
code

sc create EchoDriver type= kernel binPath= C:\path\to\echo.sys
sc start EchoDriver
user_echo.exe "Hello from userland!"
sc stop EchoDriver
sc delete EchoDriver
```

Vérification avec WinDBG

Une fois le driver chargé, vérifier dans WinDBG :

```
code

kd> !drvobj \Driver\EchoDriver 2
kd> !devobj \Device\EchoDriver
```

Observer les MajorFunction enregistrées.

Q3 : Posez un breakpoint dans WinDBG sur votre dispatch function. Envoyez un IOCTL depuis `user_echo.exe`. Faites une capture d'écran de WinDBG montrant l'IRP intercepté et le contenu du buffer (`dt _IRP` et `db` sur le SystemBuffer). Incluez cette capture dans votre rapport.

Exercice 2 : MiniFilter (2h)

Contexte

Vous allez implémenter un minifilter qui logue les accès fichiers (ouverture de fichiers) dans le kernel, observable via DbgView ou WinDBG.

Objectifs

Vous devez implémenter :

1. `filter.c` — Un minifilter qui :
   - S'enregistre auprès du Filter Manager via `FltRegisterFilter`
   - Intercepte les opérations `IRP_MJ_CREATE` (ouverture de fichiers)
   - Pour chaque fichier ouvert, affiche son chemin complet via `DbgPrint`
   - Implémente `FilterUnload` pour le déchargement propre
2. `filter.reg` — Fichier registre définissant le service, l'altitude (`370000`) et l'instance par défaut

Q4 : Qu'est-ce que l'altitude d'un minifilter ? Pourquoi les EDR utilisent-ils des altitudes élevées (300000–400000) ? Que se passerait-il si deux minifilters avaient la même altitude ?

Indications

Structure d'un minifilter

Un minifilter nécessite :

- Un tableau `FLT_OPERATION_REGISTRATION` listant les IRP interceptés et les callbacks associées
- Une structure `FLT_REGISTRATION` référençant ce tableau et la fonction d'unload
- L'appel à `FltRegisterFilter` + `FltStartFiltering` dans `DriverEntry`
- L'appel à `FltUnregisterFilter` dans `FilterUnload`

Récupérer le nom du fichier

Dans votre `PreCreateCallback`, utilisez `FltGetFileNameInformation` avec le flag `FLT_FILE_NAME_NORMALIZED` pour obtenir le chemin complet. N'oubliez pas :

- `FltParseFileNameInformation` pour parser le nom
- `FltReleaseFileNameInformation` pour libérer la structure (sinon fuite mémoire)

Fichier .reg

Le fichier registre doit définir :

- Le service sous `HKLM\SYSTEM\CurrentControlSet\Services\<NomFilter>`
- Le type `dword:00000002` (FS minifilter)
- L'image path pointant vers `\SystemRoot\System32\drivers\<NomFilter>.sys`
- L'instance par défaut avec l'altitude choisie

Chargement

```
code

regedit /s filter.reg
copy MyFilter.sys C:\Windows\System32\drivers\
fltmc load MyFilter
fltmc filters          (vérifier la présence)
fltmc unload MyFilter
```

Q5 : Chargez votre minifilter, ouvrez quelques fichiers (Notepad, Explorer), puis observez les logs dans DbgView. Combien d'opérations `IRP_MJ_CREATE` sont générées par la simple ouverture d'un fichier texte dans Notepad ? Expliquez pourquoi ce nombre est supérieur à 1.

Vérification avec WinDBG

```
code

kd> !fltkd.filters
kd> !fltkd.volumes
```

Observer votre filtre dans la liste, son altitude, et les volumes auxquels il est attaché.

Exercice 3 (bonus) : Minifilter bloquant (30min)

Modifiez votre minifilter pour **bloquer** l'accès à tout fichier dont le nom contient le mot-clé `"secret"`. Le minifilter doit :

- Retourner `FLT_PREOP_COMPLETE` avec `STATUS_ACCESS_DENIED` dans `Data->IoStatus.Status`
- Logger le blocage via `DbgPrint`

Q6 (bonus) : Testez en créant un fichier `C:\secret_data.txt`. Vérifiez que `type C:\secret_data.txt` est bloqué. Que se passe-t-il si vous tentez d'accéder au fichier via son chemin court (8.3) ? Faites une capture d'écran du blocage.

Barème indicatif

Élément	Points
Exercice 1 — Echo Driver fonctionnel (Q1–Q3)	10
Exercice 2 — MiniFilter fonctionnel (Q4–Q5)	7
Bonus — MiniFilter bloquant (Q6)	3
Total	**20**

Troubleshooting

Problème	Solution
`sc start` échoue avec STATUS_IMAGE_ALREADY_LOADED	Un driver du même nom est déjà chargé — `sc stop` puis `sc delete` d'abord
BSOD au chargement du driver	Vérifier l'appel à `IoCompleteRequest` dans chaque chemin de la dispatch function
`fltmc load` échoue	Vérifier que le `.reg` a été importé et que le `.sys` est dans `System32\drivers`
DeviceIoControl retourne ERROR_INVALID_FUNCTION	Le `CTL_CODE` du userland ne correspond pas à celui du driver — vérifier le header partagé
DbgPrint n'affiche rien	En kernel, utiliser `DbgPrintEx(DPFLTR_DEFAULT_ID, DPFLTR_ERROR_LEVEL, ...)` pour forcer l'affichage